

2 - DNS-Lab-Server-Deployment-Delegation

Performed by *Nikita Niakhai*

Part 1: Installing BIND9 DNS Server

Task 1.1: Install BIND9 Package

1. Downloaded, installed ISO for Ubuntu servers 24.04.3 LTS []. Named machines:
 - a. `ns1` - Primary DNS server -
 - i. IP: `192.168.100.10`
 - b. `ns2` - Secondary DNS server -
 - i. IP: `192.168.100.11`
 - c. `ns3` - Delegated subdomain DNS server -
 - i. IP: `192.168.100.12`
 - d. 11 GB disk space, 3164 MB RAM, 1 processor
2. Configured Network settings on each machine to use **Adapter 1: Host-only Adapter (VBoxNet0)**.
 - a. Alternately, to access Internet I will use **NAT adapter**
3. Configured bidirectional copy-paste on all machines.
4. Updated packages and installed needed ones.
 - a. Had problem accessing root user via `su -`, but tried `sudo -i` and it worked.

```
# Entered root user
sudo -i
```

```
# Update package repositories

sudo apt update && apt upgrade -y && reboot

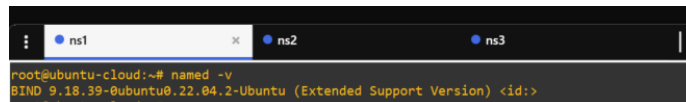
# Install BIND9 and related utilities

apt --fix-broken install

sudo apt install bind9 bind9utils bind9-doc dnsutils -y

# Verify BIND9 installation

named -v
```

A terminal window with three tabs labeled ns1, ns2, and ns3. The active tab is ns1. The terminal shows the command 'named -v' being executed, resulting in the output 'BIND 9.18.39-0ubuntu0.22.04.2-Ubuntu (Extended Support Version) <id>'. The prompt is 'root@ubuntu-cloud:~#'.

Expected Output: BIND version information should be displayed (e.g., BIND 9.18.x).

 Problem 1: After executing default updates and upgrades my VMs stop working, black screens. I tried reinstalling images and VMs with more disk space, memory - but the same error.

Solution: I gave up using Virtual Box and opened GNS3 with GNS3 VM where I deployed 3 clients, connected them to 2 switches , figure 1,2.

ens3 - to access to enternet (interface switch1)

ens4 - to access LAN

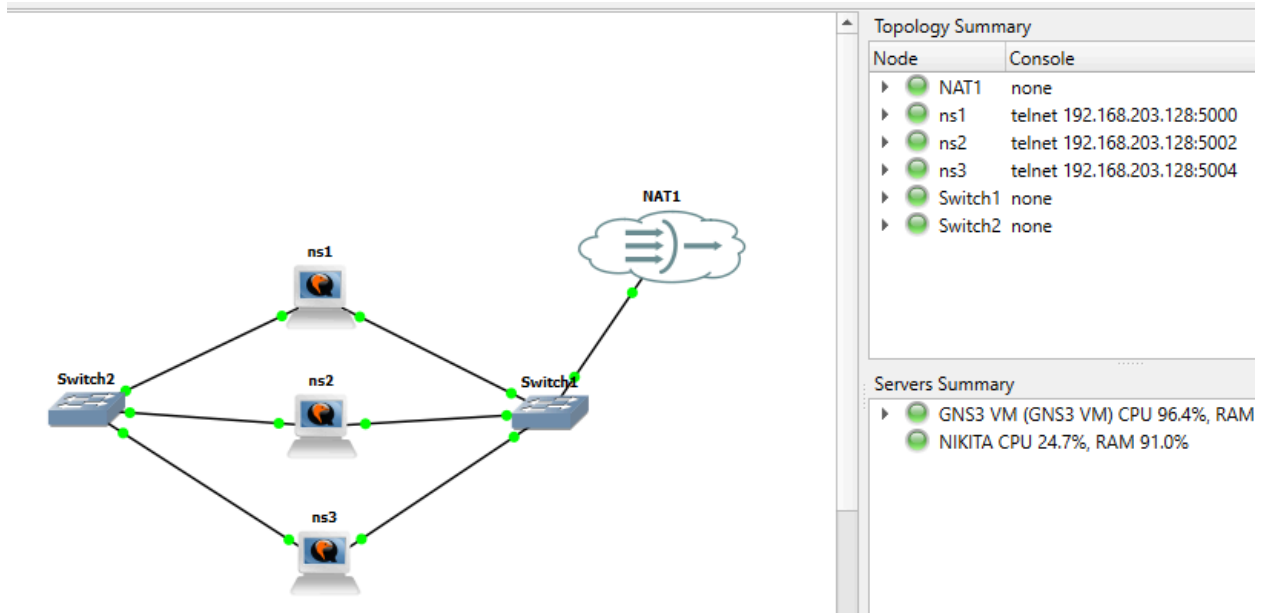


Figure 1. Topology with Internet Access

- Connected each machine to the switch, edit `interfaces` file where I added IP of my Virtual Network connected to the internet, figure 3,4.

```

GNU nano 6.2 /etc/network/interfaces
192.168.56.1

```

Figure 3. Configuration on each machine to access internet, part 1

```
ubuntu@ubuntu-cloud:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 0c:43:9f:f1:00:00 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 192.168.174.130/24 metric 100 brd 192.168.174.255 scope global dynamic ens3
        valid_lft 1740sec preferred_lft 1740sec
    inet6 fe80::e43:9fff:fef1:0/64 scope link
        valid_lft forever preferred_lft forever
```

Figure 4. Configuration on each machine to access internet, part 2



Problem 2: No space available on nodes.

`lsblk` is used to check disk space

Solution: Increased RAM to 2048 MB and Disk Space to 11gb on each node, rebooted. Increase memory on GNS3VM

5. Configured static IP addresses using netplan

```
sudo nano /etc/netplan/01-netcfg.yaml
```

```
network:
  version: 2
  ethernets:
    ens3:
      dhcp4: yes
    ens4:
      dhcp4: no
  addresses:
    - 192.168.100.10/24
```

```
sudo chmod 600 /etc/netplan/01-netcfg.yaml
```

```
sudo netplan apply
```

Task 1.2: Understanding BIND9 Directory Structure

Task 1.3: Configure DNS Server Options

Edit the options configuration file on **Server 1, figure 5**:

```
acl "trusted" {  
    192.168.100.0/24; # Allow queries from local network  
    localhost;  
    localnets;  
};  
  
options {  
    directory "/var/cache/bind";  
  
    # Allow queries from trusted networks only  
    allow-query { trusted; };  
  
    # Disable recursion for authoritative-only server  
  
    recursion no;  
  
    # Listen on IPv4 only  
    listen-on { 192.168.100.10; 127.0.0.1; };
```

```

listen-on-v6 { any; };

# Disable zone transfers by default
allow-transfer { none ; };

# DNSSEC validation
dnssec-validation auto;

# Forwarders (optional, for external queries)
forwarders {
    8.8.8.8;
    8.8.4.4;
};

};

```

```

ubuntu@ubuntu-cloud:~$ sudo cat /etc/bind/named.conf.options
acl "trusted" {
    192.168.100.0/24; # Allow queries from local network
    localhost;
    localnets;
};

options {
    directory "/var/cache/bind";

    # Allow queries from trusted networks only
    allow-query { trusted; };

    # Disable recursion for authoritative-only server
    recursion no;

    # Listen on IPv4 only
    listen-on { 192.168.100.10; 127.0.0.1; };

    # Disable zone transfers by default
    allow-transfer { none ; };

    # DNSSEC validation
    dnssec-validation auto;

    # Forwarders (optional, for external queries)
    forwarders {
        8.8.8.8;
        8.8.4.4;
    };
};

```

Figure 5. Configuration for DNS server (`ns1`) config

Task 1.4: Verify Configuration Syntax, figure 6

```

ubuntu@ubuntu-cloud:~$ sudo named-checkconf
ubuntu@ubuntu-cloud:~$ sudo named-checkconf -p
acl "trusted" {
    192.168.100.0/24;
    "localhost";
    "localnets";
};
options {
    directory "/var/cache/bind";
    listen-on {
        192.168.100.10/32;
        127.0.0.1/32;
    };
    dnssec-validation auto;
    recursion no;
    allow-query {
        "trusted";
    };
    allow-transfer {
        "none";
    };
    forwarders {
        8.8.8.8;
        8.8.4.4;
    };
};
zone "." {
    type hint;
    file "/usr/share/dns/root.hints";
};
zone "localhost" {
    type master;
    file "/etc/bind/db.local";
};
zone "127.in-addr.arpa" {
    type master;
    file "/etc/bind/db.127";
};
zone "0.in-addr.arpa" {
    type master;
    file "/etc/bind/db.0";
};
zone "255.in-addr.arpa" {
    type master;
    file "/etc/bind/db.255";
};

```

Figure 6. Check for syntax error: no errors are displayed, the configuration is syntactically correct.

Part 2: Creating Primary DNS Zone

Task 2.1,2.2: Define Forward and Reverse Zones

Edit the local zones configuration on **Server 1**:

```
zone "example.lab" {
    type master;
    file "/etc/bind/zones/db.example.lab";
    allow-transfer { 192.168.100.11; }; # Allow transfer to ns2
    notify yes;
};

zone "100.168.192.in-addr.arpa" {
    #reverse lookup zone

    type master;
    file "/etc/bind/zones/db.192.168.100";
    allow-transfer { 192.168.100.11; };
    notify yes;
};
```



```
GNU nano 6.2 /etc/bind/named.conf.local
zone "example.lab" {
    type master;
    file "/etc/bind/zones/db.example.lab";
    allow-transfer { 192.168.100.11; }; # Allow transfer to ns2
    notify yes;
};

zone "100.168.192.in-addr.arpa" {
    #reverse lookup zone

    type master;
    file "/etc/bind/zones/db.192.168.100";
    allow-transfer { 192.168.100.11; };
    notify yes;
};
```

Figure 7. zone config file

Task 2.3: Created Zone Files Directory

Task 2.4: Create Forward Zone File

Create and edit the forward zone file:


```
sudo nano /etc/bind/zones/db.example.lab
```

Add the following content:

```
$TTL 86400
@ IN SOA ns1.example.lab. admin.example.lab. (
    2025120201 ; Serial (YYYYMMDDNN)
    3600      ; Refresh (1 hour)
    1800      ; Retry (30 minutes)
    604800    ; Expire (1 week)
    86400 )   ; Negative Cache TTL (1 day)

; Name Servers
@ IN NS ns1.example.lab.
@ IN NS ns2.example.lab.

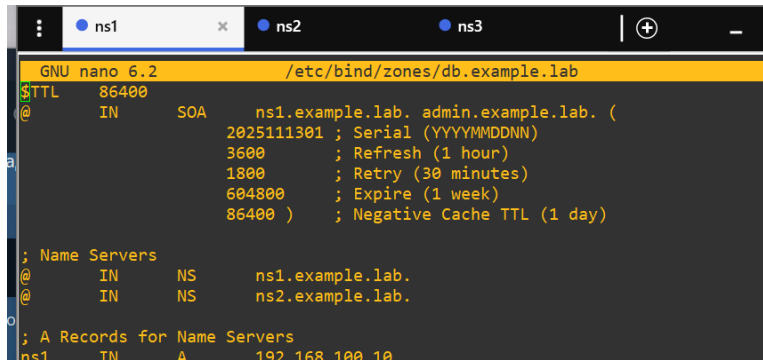
; A Records for Name Servers
ns1 IN A 192.168.100.10
ns2 IN A 192.168.100.11

; Mail Exchange Record
@ IN MX 10 mail.example.lab.

; Host A Records
www IN A 192.168.100.20
mail IN A 192.168.100.25
ftp IN A 192.168.100.30
db IN A 192.168.100.35

; Alias (CNAME) Records
webmail IN CNAME www.example.lab.
smtp IN CNAME mail.example.lab.

; Delegation for subdomain
dept IN NS ns3.example.lab.
ns3 IN A 192.168.100.12
```

A screenshot of a terminal window with three tabs labeled 'ns1', 'ns2', and 'ns3'. The active tab is 'ns1'. The terminal shows the GNU nano 6.2 editor editing the file /etc/bind/zones/db.example.lab. The content of the file is a DNS zone configuration for 'example.lab'. It starts with a \$TTL of 86400, followed by an SOA record for ns1.example.lab. with serial 2025111301, refresh 3600, retry 1800, expire 604800, and negative cache TTL 86400. Then it lists Name Servers: ns1.example.lab. and ns2.example.lab. Finally, it shows A Records for Name Servers: ns1 with IP 192.168.100.10.

```
GNU nano 6.2 /etc/bind/zones/db.example.lab
$TTL 86400
@      IN      SOA      ns1.example.lab. admin.example.lab. (
                                2025111301 ; Serial (YYYYMMDDNN)
                                3600      ; Refresh (1 hour)
                                1800      ; Retry (30 minutes)
                                604800    ; Expire (1 week)
                                86400    ) ; Negative Cache TTL (1 day)

; Name Servers
@      IN      NS       ns1.example.lab.
@      IN      NS       ns2.example.lab.

; A Records for Name Servers
ns1    IN      A        192.168.100.10
```

Figure 8. forward zone config file

\$TTL: Default time-to-live for records (in seconds)

SOA Record: Start of Authority - defines zone properties

NS Records: Nameserver records identifying authoritative servers

A Records: Maps hostnames to IPv4 addresses

MX Record: Mail exchange server with priority

CNAME Records: Canonical name (alias) records

Delegation Records: NS and glue A records for subdomain delegation

Task 2.5: Create Reverse Zone File

```
sudo nano /etc/bind/zones/db.192.168.100
```

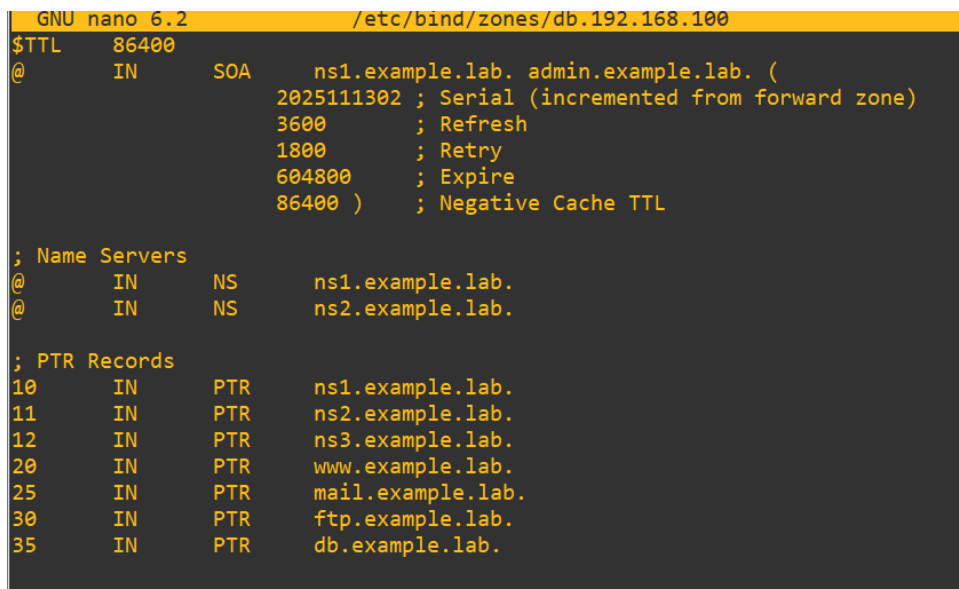
Add the following content:

```
$TTL 86400
@      IN      SOA      ns1.example.lab. admin.example.lab. (
                                2025120201 ; Serial (incremented from forward zone)
                                3600      ; Refresh
                                1800      ; Retry
                                604800    ; Expire
                                86400    ) ; Negative Cache TTL

; Name Servers
@      IN      NS       ns1.example.lab.
@      IN      NS       ns2.example.lab.
```

; PTR Records

```
10 IN PTR ns1.example.lab.
11 IN PTR ns2.example.lab.
12 IN PTR ns3.example.lab.
20 IN PTR www.example.lab.
25 IN PTR mail.example.lab.
30 IN PTR ftp.example.lab.
35 IN PTR db.example.lab.
```



The screenshot shows a terminal window with the GNU nano 6.2 text editor open to the file /etc/bind/zones/db.192.168.100. The file contains a reverse zone configuration for the 192.168.100.0/24 network. It starts with a \$TTL of 86400 and an SOA record for ns1.example.lab. with serial 2025111302. It then lists name servers ns1.example.lab. and ns2.example.lab. Finally, it contains a section for PTR records mapping IP addresses to hostnames.

```
GNU nano 6.2 /etc/bind/zones/db.192.168.100
$TTL      86400
@         IN      SOA      ns1.example.lab. admin.example.lab. (
                        2025111302 ; Serial (incremented from forward zone)
                        3600      ; Refresh
                        1800      ; Retry
                        604800    ; Expire
                        86400 )   ; Negative Cache TTL

; Name Servers
@         IN      NS       ns1.example.lab.
@         IN      NS       ns2.example.lab.

; PTR Records
10        IN      PTR      ns1.example.lab.
11        IN      PTR      ns2.example.lab.
12        IN      PTR      ns3.example.lab.
20        IN      PTR      www.example.lab.
25        IN      PTR      mail.example.lab.
30        IN      PTR      ftp.example.lab.
35        IN      PTR      db.example.lab.
```

Figure 9. reverse zone config file

Task 2.6: Validate Zone Files

Check zone file syntax using `named-checkzone`, figure 11,12:

Check forward zone

```
sudo named-checkzone example.lab /etc/bind/zones/db.example.lab
```

Check reverse zone

```
sudo named-checkzone 100.168.192.in-addr.arpa /etc/bind/zones/db.192.168.100
```

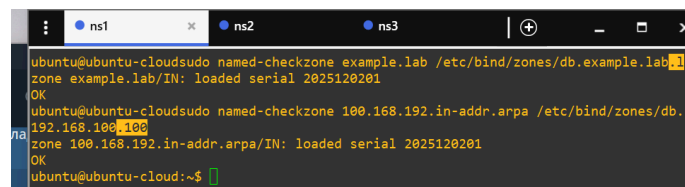
```

1. ubuntu@ubuntu-cloudsudo named-checkzone example.lab /etc/bind/zones/db.example.lab
zone example.lab/IN: loaded serial 2025111301
OK
ubuntu@ubuntu-cloud:~$ sudo named-checkzone 100.168.192.in-addr.arpa /etc/bind/zones/db.192.168.100
zone 100.168.192.in-addr.arpa/IN: loaded serial 2025111302
OK

```

Figure 10. Validation log part 1

I fixed Serial to current date and fixed error, figure 11, 12.



```

ns1 ns2 ns3
ubuntu@ubuntu-cloudsudo named-checkzone example.lab /etc/bind/zones/db.example.lab
zone example.lab/IN: loaded serial 2025120201
OK
ubuntu@ubuntu-cloudsudo named-checkzone 100.168.192.in-addr.arpa /etc/bind/zones/db.192.168.100
zone 100.168.192.in-addr.arpa/IN: loaded serial 2025120201
OK
ubuntu@ubuntu-cloud:~$

```

Figure 11. Validation log part 1

Task 2.7: Restart BIND9 Service

Restart the service

```
sudo systemctl restart bind9
```

Check service status

```
sudo systemctl status bind9
```

Enable autostart on boot

```

ubuntu@ubuntu-cloud:~$ sudo systemctl restart bind9
ubuntu@ubuntu-cloud:~$ sudo systemctl status bind9
● named.service - BIND Domain Name Server
   Loaded: loaded (/lib/systemd/system/named.service; enabled; vendor preset:
   Active: active (running) since Tue 2025-12-02 17:50:29 UTC; 4s ago
     Docs: man:named(8)
  Process: 1106 ExecStart=/usr/sbin/named $OPTIONS (code=exited, status=0/SUC
 Main PID: 1108 (named)
    Tasks: 4 (limit: 2307)
   Memory: 22.1M
      CPU: 123ms
   CGroup: /system.slice/named.service
           └─1108 /usr/sbin/named -u bind

Dec 02 17:50:29 ubuntu-cloud named[1108]: REFUSED unexpected RCODE resolving '.>
Dec 02 17:50:29 ubuntu-cloud named[1108]: REFUSED unexpected RCODE resolving '.>
Dec 02 17:50:29 ubuntu-cloud named[1108]: REFUSED unexpected RCODE resolving '.>
Dec 02 17:50:29 ubuntu-cloud named[1108]: REFUSED unexpected RCODE resolving '.>
Dec 02 17:50:29 ubuntu-cloud named[1108]: REFUSED unexpected RCODE resolving '.>
Dec 02 17:50:29 ubuntu-cloud named[1108]: REFUSED unexpected RCODE resolving '.>
Dec 02 17:50:29 ubuntu-cloud named[1108]: REFUSED unexpected RCODE resolving '.>
Dec 02 17:50:29 ubuntu-cloud named[1108]: REFUSED unexpected RCODE resolving '.>
Dec 02 17:50:29 ubuntu-cloud named[1108]: REFUSED unexpected RCODE resolving '.>
Dec 02 17:50:29 ubuntu-cloud named[1108]: resolver priming query complete: fail>

```

Figure 12. Refresh the bind9 server - command log

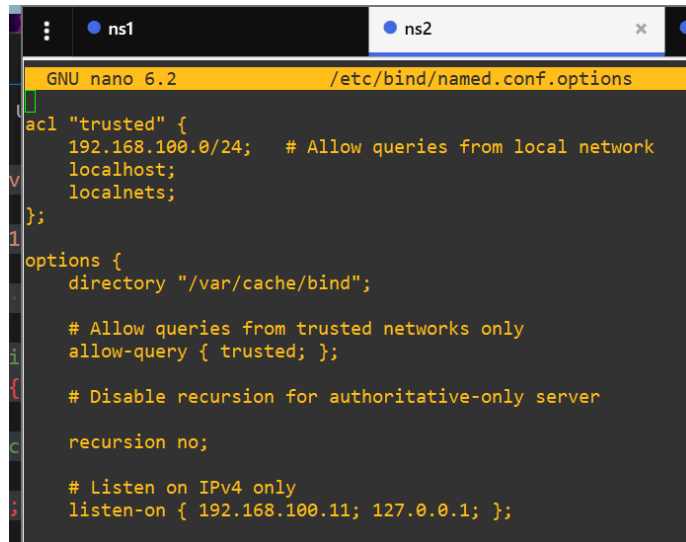
Part 3: Configuring Secondary (Slave) DNS Server

Task 3.1: Configure Secondary Server Options

On **Server 2 (ns2)**, edit the options file:

```
sudo nano /etc/bind/named.conf.options
```

I adjusted the listen-on address, figure 13.



```
GNU nano 6.2 /etc/bind/named.conf.options
acl "trusted" {
    192.168.100.0/24; # Allow queries from local network
    localhost;
    localnets;
};

options {
    directory "/var/cache/bind";

    # Allow queries from trusted networks only
    allow-query { trusted; };

    # Disable recursion for authoritative-only server
    recursion no;

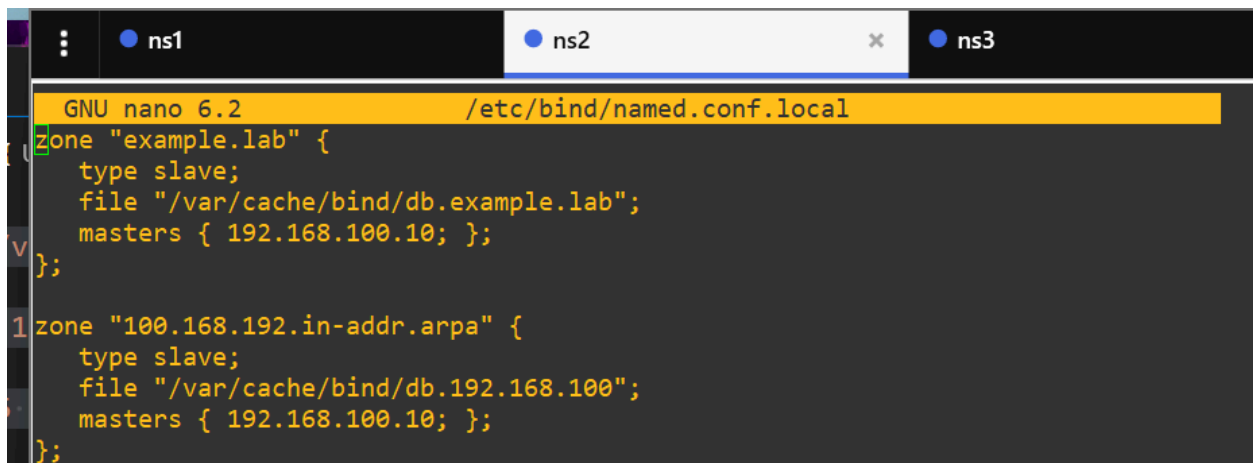
    # Listen on IPv4 only
    listen-on { 192.168.100.11; 127.0.0.1; };
```

Figure 13. Server 2 - config for DNS

Task 3.2: Define Secondary Zones

Edit the local zones configuration:

```
sudo nano /etc/bind/named.conf.local
```



```
GNU nano 6.2 /etc/bind/named.conf.local
zone "example.lab" {
    type slave;
    file "/var/cache/bind/db.example.lab";
    masters { 192.168.100.10; };
};

zone "100.168.192.in-addr.arpa" {
    type slave;
    file "/var/cache/bind/db.192.168.100";
    masters { 192.168.100.10; };
};
```

Figure 14. Server 2 - local zone config

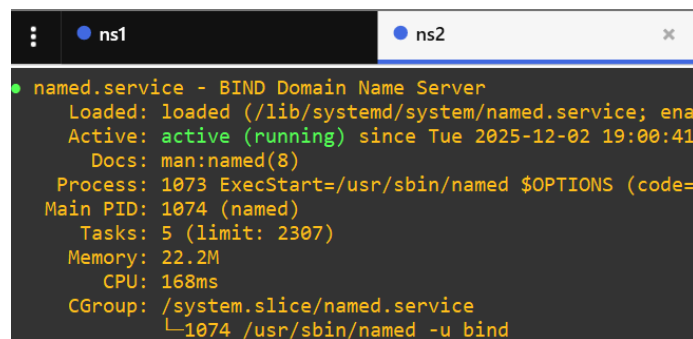
```

zone "example.lab" {
    type slave;
    file "/var/cache/bind/db.example.lab";
    masters { 192.168.100.10; };
};

zone "100.168.192.in-addr.arpa" {
    type slave;
    file "/var/cache/bind/db.192.168.100";
    masters { 192.168.100.10; };
};

```

Task 3.3: Restart Secondary Server



```

ns2
● named.service - BIND Domain Name Server
   Loaded: loaded (/lib/systemd/system/named.service; enabled; vendor preset: enabled)
   Active: active (running) since Tue 2025-12-02 19:00:41; 1min 45s ago
     Docs: man:named(8)
   Process: 1073 ExecStart=/usr/sbin/named $OPTIONS (code=exited, status=0/SUCCESS)
    Main PID: 1074 (named)
      Tasks: 5 (limit: 2307)
     Memory: 22.2M
        CPU: 168ms
    CGroup: /system.slice/named.service
            └─1074 /usr/sbin/named -u bind

```

Figure 15. BIND9 service is running on server 2

Task 3.4: Verify Zone Transfer

I Check that zones were transferred successfully, figure 16:

Check syslog for transfer messages

```
sudo tail -f /var/log/syslog | grep named
```

Verify zone files were created

```
ls -la /var/cache/bind/
```

The image shows a terminal window with three tabs labeled 'ns1', 'ns2', and 'ns3'. The 'ns2' tab is active. The terminal output shows a series of log messages from the 'named' service on 'ubuntu-cloud'. The logs indicate a zone transfer from 192.168.100.10#53 to 192.168.100.10#53 for the 'example.lab/IN' zone. The transfer is successful, with 15 records, 351 bytes, and 0.001 seconds. The logs also show a resolver priming query complete failure. Below the logs, the user runs the command 'ls -la /var/cache/bind/' and the output shows the contents of the directory, including files like 'db.192.168.100', 'db.example.lab', 'managed-keys.bind', and 'managed-keys.bind.jnl'.

```
root@ubuntu-cloud:~# sudo tail -f /var/log/syslog | grep named
Dec 2 19:00:41 ubuntu-cloud named[1074]: REFUSED unexpected RCODE resolving './NS/IN': 198.41.0.4#53
Dec 2 19:00:41 ubuntu-cloud named[1074]: REFUSED unexpected RCODE resolving './NS/IN': 192.112.36.4#53
Dec 2 19:00:41 ubuntu-cloud named[1074]: REFUSED unexpected RCODE resolving './NS/IN': 198.97.190.53#53
Dec 2 19:00:41 ubuntu-cloud named[1074]: resolver priming query complete: failure
Dec 2 19:00:41 ubuntu-cloud named[1074]: zone example.lab/IN: Transfer started.
Dec 2 19:00:41 ubuntu-cloud named[1074]: transfer of 'example.lab/IN' from 192.168.100.10#53: connected using 192.168.100.10#53
Dec 2 19:00:41 ubuntu-cloud named[1074]: zone example.lab/IN: transferred serial 2025120201
Dec 2 19:00:41 ubuntu-cloud named[1074]: transfer of 'example.lab/IN' from 192.168.100.10#53: Transfer status: success
Dec 2 19:00:41 ubuntu-cloud named[1074]: transfer of 'example.lab/IN' from 192.168.100.10#53: Transfer completed: 1 messages, 15 records, 351 bytes, 0.001 secs (351000 bytes/sec) (serial 2025120201)
Dec 2 19:00:41 ubuntu-cloud named[1074]: zone example.lab/IN: sending notifies (serial 2025120201)
^Croot@ubuntu-cloud:~# ls -la /var/cache/bind/
total 24
drwxrwxr-x 2 root bind 4096 Dec 2 19:01 .
drwxr-xr-x 12 root root 4096 Dec 2 16:20 ..
-rw-r--r-- 1 bind bind 688 Dec 2 19:00 db.192.168.100
-rw-r--r-- 1 bind bind 717 Dec 2 19:00 db.example.lab
-rw-r--r-- 1 bind bind 1421 Dec 2 19:01 managed-keys.bind
-rw-r--r-- 1 bind bind 2248 Dec 2 19:00 managed-keys.bind.jnl
```

Figure 16

Task 3.5: Manual Zone Transfer Testing

From **Server 2**, I manually requested a zone transfer, figure 17:

```
dig @192.168.100.10 example.lab AXFR
```

AXFR (Full Zone Transfer): Transfers the complete zone file from master to slave


```
root@ubuntu-cloud:~# dig @192.168.100.10 example.lab AXFR

; <<>> DiG 9.18.39-0ubuntu0.22.04.2-Ubuntu <<>> @192.168.100.10 example.lab AXFR
; (1 server found)
;; global options: +cmd
example.lab.      86400    IN      SOA     ns1.example.lab. admin.example.lab. 2025120201 3600 1800 604800 86400
example.lab.      86400    IN      NS      ns1.example.lab.
example.lab.      86400    IN      NS      ns2.example.lab.
example.lab.      86400    IN      MX      10 mail.example.lab.
db.example.lab.   86400    IN      A       192.168.100.35
dept.example.lab. 86400    IN      NS      ns3.example.lab.
ftp.example.lab.  86400    IN      A       192.168.100.30
mail.example.lab. 86400    IN      A       192.168.100.25
ns1.example.lab.  86400    IN      A       192.168.100.10
ns2.example.lab.  86400    IN      A       192.168.100.11
ns3.example.lab.  86400    IN      A       192.168.100.12
smtp.example.lab. 86400    IN      CNAME   mail.example.lab.
webmail.example.lab. 86400    IN      CNAME   www.example.lab.
www.example.lab.  86400    IN      A       192.168.100.20
example.lab.      86400    IN      SOA     ns1.example.lab. admin.example.lab. 2025120201 3600 1800 604800 86400
;; Query time: 4 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (TCP)
;; WHEN: Tue Dec 02 19:06:31 UTC 2025
;; XFR size: 15 records (messages 1, bytes 390)
```

Figure 17. Log for transferring complete zone file on server 2 from server 1 (master)

Part 4: Implementing DNS Delegation

Task 4.1: Understanding Delegation Concepts

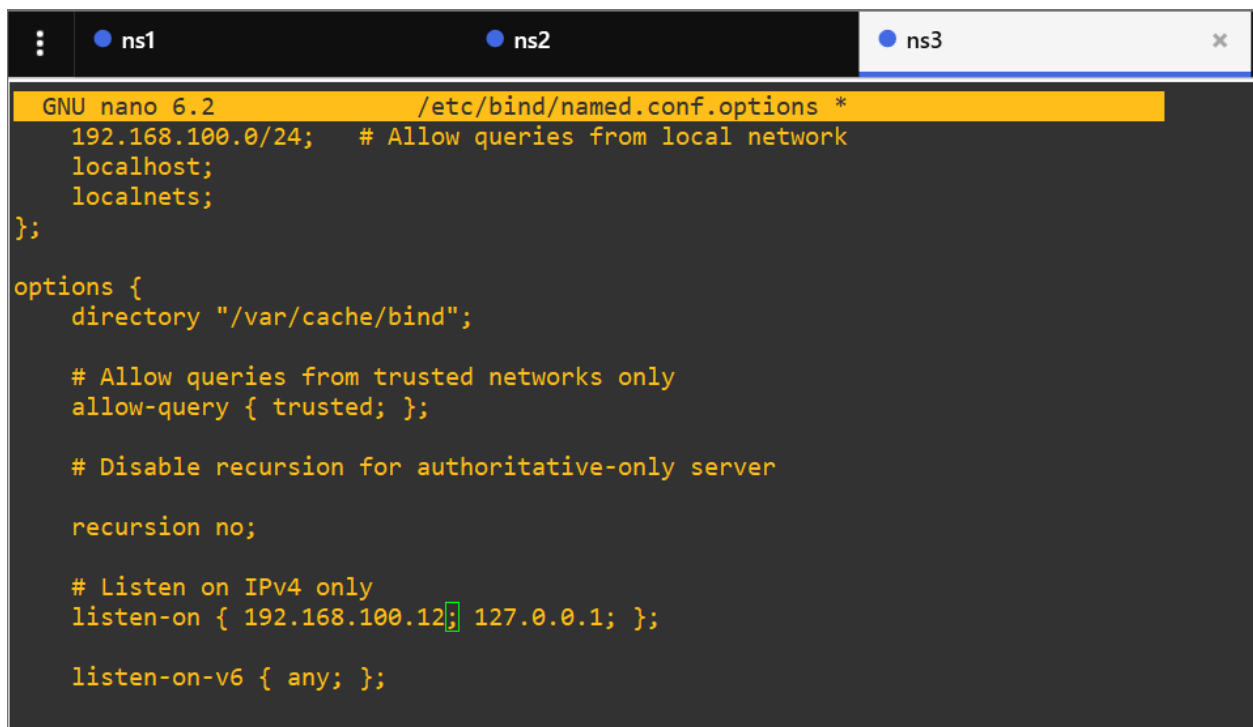
Task 4.2: Install BIND9 on Delegated Server, figure 18

```
ubuntu@ubuntu-cloud:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 0c:5c:07:4a:00:00 brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    inet 192.168.122.119/24 metric 100 brd 192.168.122.255 scope global dynamic ens3
        valid_lft 3223sec preferred_lft 3223sec
    inet6 fe80::e5c:7ff:fe4a:0/64 scope link
        valid_lft forever preferred_lft forever
3: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 0c:5c:07:4a:00:01 brd ff:ff:ff:ff:ff:ff
    altname enp0s4
    inet 192.168.100.12/24 brd 192.168.100.255 scope global ens4
        valid_lft forever preferred_lft forever
    inet6 fe80::e5c:7ff:fe4a:1/64 scope link
        valid_lft forever preferred_lft forever
ubuntu@ubuntu-cloud:~$ named -v
BIND 9.18.39-0ubuntu0.22.04.2-Ubuntu (Extended Support Version) <id>
```

Figure 18. Configuration for server 3 (ip on ens4 / bind server)

Task 4.3: Configure Delegated Subdomain Zone

On **Server 3**, I edited the options file, figure 19.



```
GNU nano 6.2 /etc/bind/named.conf.options *
192.168.100.0/24; # Allow queries from local network
localhost;
localnets;
};

options {
    directory "/var/cache/bind";

    # Allow queries from trusted networks only
    allow-query { trusted; };

    # Disable recursion for authoritative-only server
    recursion no;

    # Listen on IPv4 only
    listen-on { 192.168.100.12; 127.0.0.1; };

    listen-on-v6 { any; };
```

Figure 19. Subdomain config - server 3

Task 4.4: Define Subdomain Zone

I edited the local zone file:

```
sudo nano /etc/bind/named.conf.local
```

Added the delegated subdomain zone, figure 20.

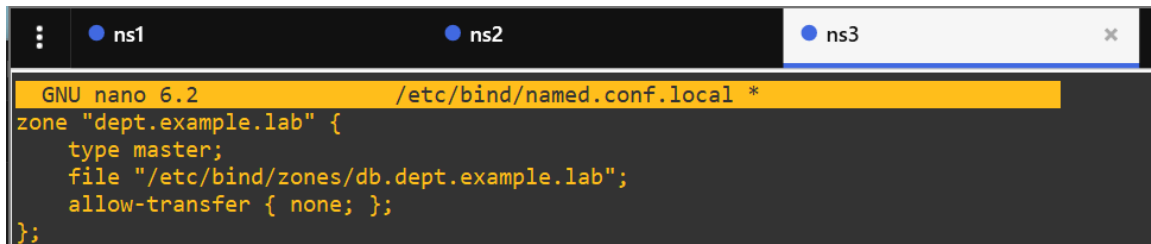
A terminal window with three tabs labeled ns1, ns2, and ns3. The ns3 tab is active. The terminal shows the GNU nano 6.2 editor editing /etc/bind/named.conf.local. The content of the file is: zone "dept.example.lab" { type master; file "/etc/bind/zones/db.dept.example.lab"; allow-transfer { none; }; };

Figure 20. Subdomain local zone config file - server 3

Task 4.5: Create Subdomain Zone File

```
sudo mkdir -p /etc/bind/zones
```

```
sudo nano /etc/bind/zones/db.dept.example.lab
```

I added the subdomain zone content:

```
$TTL 86400
@ IN SOA ns3.example.lab. admin.dept.example.lab. (
    2025120201 ; Serial
    3600      ; Refresh
    1800      ; Retry
    604800    ; Expire
    86400 )   ; Negative Cache TTL

; Name Server for the subdomain
@ IN NS ns3.example.lab.

; A record for the name server itself (required in child zone)
ns3.example.lab. IN A 192.168.100.12

; Subdomain hosts
hr IN A 192.168.100.50
finance IN A 192.168.100.51
it IN A 192.168.100.52
admin IN A 192.168.100.53
```

```
; CNAME record
portal IN CNAME hr.dept.example.lab.
```

```
onf                                sudo cat /etc/bind/zones/db.dept.example.lab
$TTL      86400
@          IN      SOA      ns3.example.lab. admin.dept.example.lab. (
                                2025120201 ; Serial
                                3600       ; Refresh
                                1800       ; Retry
                                604800    ; Expire
                                86400    ) ; Negative Cache TTL
; Name Server for the subdomain
@          IN      NS       ns3.example.lab.
; A record for the name server itself (required in child zone)
ns3.example.lab. IN    A      192.168.100.12
; Subdomain hosts
hr         IN      A        192.168.100.50
finance IN      A        192.168.100.51
it         IN      A        192.168.100.52
admin     IN      A        192.168.100.53
; CNAME record
portal IN      CNAME      hr.dept.example.lab.
```

Figure 21. Configuration for subdomain zone file on server 3

Task 4.6: Validate and Restart

I performed following steps:

- # Check zone file

```
sudo named-checkzone dept.example.lab /etc/bind/zones/db.dept.example.lab
```

- # Check configuration

```
sudo named-checkconf
```

- # Restart service

```
sudo systemctl restart bind9
```

Validation was successful, the warning bellow is unharmlful, because this is the subdomain.

```

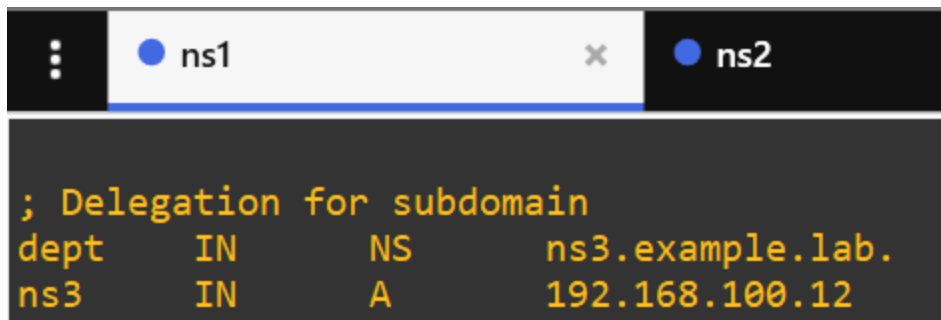
ubuntu@ubuntu-cloud:~$ sudo named-checkzone dept.example.lab /etc/bind/zones/db.dept.example.lab
/etc/bind/zones/db.dept.example.lab:13: ignoring out-of-zone data (ns3.example.lab)
zone dept.example.lab/IN: loaded serial 2025120201
OK
ubuntu@ubuntu-cloud:~$ sudo named-checkconf
ubuntu@ubuntu-cloud:~$ sudo systemctl restart bind9
ubuntu@ubuntu-cloud:~$ sudo systemctl status bind9
● named.service - BIND Domain Name Server
   Loaded: loaded (/lib/systemd/system/named.service; enabled; vendor preset:
   Active: active (running) since Tue 2025-12-02 19:41:48 UTC; 5s ago
     Docs: man:named(8)
   Process: 962 ExecStart=/usr/sbin/named $OPTIONS (code=exited, status=0/SUCCE
 Main PID: 963 (named)
    Tasks: 4 (limit: 2307)
   Memory: 22.0M
      CPU: 83ms
   CGroup: /system.slice/named.service
           └─963 /usr/sbin/named -u bind

```

Figure 22. Log for configuration checks on server 3

Task 4.7: Verify Delegation in Parent Zone

Here I returned to **Server 1 (ns1)** and verified that the delegation records are present in `/etc/bind/zones/db.example.lab`:



```

; Delegation for subdomain
dept      IN      NS      ns3.example.lab.
ns3       IN      A       192.168.100.12

```

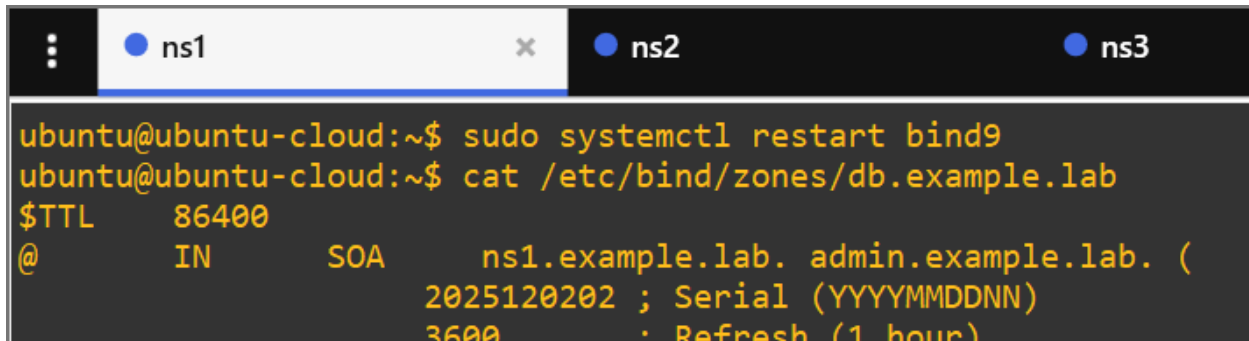
Figure 23. Log for verifying presence of delegation records

Increment the serial number in the parent zone and restart, figure 24:

```
sudo nano /etc/bind/zones/db.example.lab
```

I changed serial to **2025120202**:

```
sudo systemctl restart bind9
```

A terminal window with three tabs labeled 'ns1', 'ns2', and 'ns3'. The 'ns1' tab is active. The terminal shows the following commands and output:

```
ubuntu@ubuntu-cloud:~$ sudo systemctl restart bind9
ubuntu@ubuntu-cloud:~$ cat /etc/bind/zones/db.example.lab
$TTL      86400
@         IN      SOA      ns1.example.lab. admin.example.lab. (
                                2025120202 ; Serial (YYYYMMDDNN)
                                3600      : Refresh (1 hour)
```

Figure 24. Executed actions of incrementing the serial number in the parent zone + status of restarted bind9

Part 5: Testing and Verification

Task 5.1: Basic DNS Query Testing

I used `dig` command to test DNS resolution:

```
# Query the primary server

dig @192.168.100.10 www.example.lab

# Query the secondary server

dig @192.168.100.11 www.example.lab

# Query specific record types

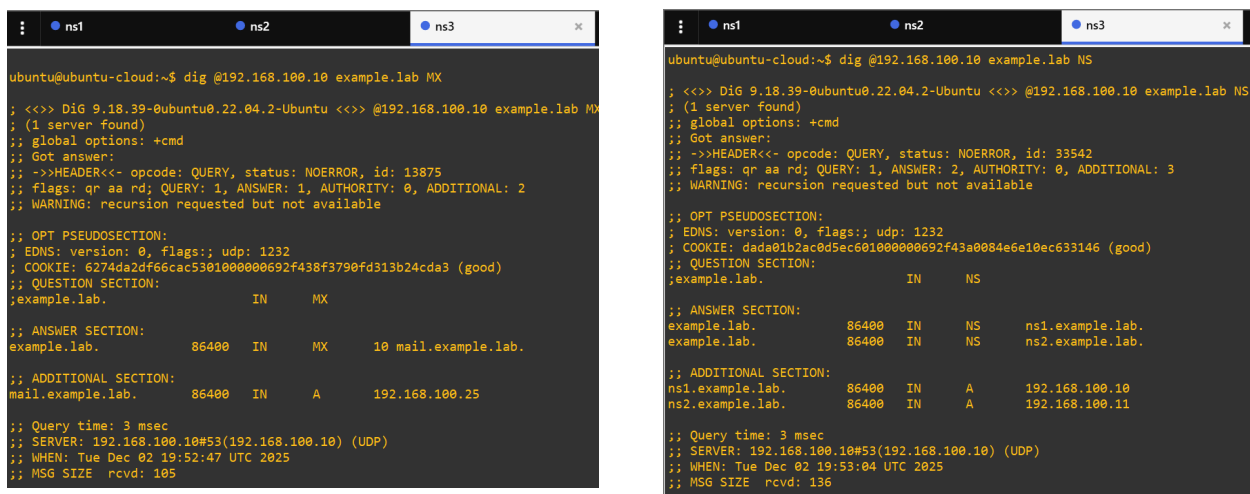
dig @192.168.100.10 example.lab MX

dig @192.168.100.10 example.lab NS

dig @192.168.100.10 example.lab SOA
```

All commands showed status NOERROR, right record types, servers found, figure 25,26,27.

Verifications using basic DNS query testing. Correct IP addresses and records are shown.



The image displays two terminal windows side-by-side, showing the output of DNS queries using the 'dig' command. The left window shows the results of a query for the MX record of 'example.lab' from server ns3. The right window shows the results of a query for the NS records of 'example.lab' from server ns3.

```
ubuntu@ubuntu-cloud:~$ dig @192.168.100.10 example.lab MX
; <<>> DiG 9.18.39-0ubuntu0.22.04.2-Ubuntu <<>> @192.168.100.10 example.lab MX
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 13875
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 2
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 6274da2df66cac5301000000692f438f3790fd313b24cda3 (good)
;; QUESTION SECTION:
;example.lab.                IN      MX

;; ANSWER SECTION:
example.lab.                86400   IN      MX      10 mail.example.lab.

;; ADDITIONAL SECTION:
mail.example.lab.          86400   IN      A        192.168.100.25

;; Query time: 3 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Tue Dec 02 19:52:47 UTC 2025
;; MSG SIZE rcvd: 105
```

```
ubuntu@ubuntu-cloud:~$ dig @192.168.100.10 example.lab NS
; <<>> DiG 9.18.39-0ubuntu0.22.04.2-Ubuntu <<>> @192.168.100.10 example.lab NS
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 33542
;; flags: qr aa rd; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 3
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: dada01b2ac8d5ec601000000692f43a0084e6e10ec633146 (good)
;; QUESTION SECTION:
;example.lab.                IN      NS

;; ANSWER SECTION:
example.lab.                86400   IN      NS      ns1.example.lab.
example.lab.                86400   IN      NS      ns2.example.lab.

;; ADDITIONAL SECTION:
ns1.example.lab.           86400   IN      A        192.168.100.10
ns2.example.lab.           86400   IN      A        192.168.100.11

;; Query time: 3 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Tue Dec 02 19:53:04 UTC 2025
;; MSG SIZE rcvd: 136
```

Figure 25. from server 3 to server 1 for MS,NS records

```

ubuntu@ubuntu-cloud:~$ dig @192.168.100.11 www.example.lab

; <<>> DiG 9.18.39-0ubuntu0.22.04.2-Ubuntu <<>> @192.168.100.11 www.example.lab
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 57272
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: b5901fb3342d86df0100000692f438723cc8784539f9aa0 (good)
;; QUESTION SECTION:
;www.example.lab.                IN      A

;; ANSWER SECTION:
www.example.lab.                86400   IN      A      192.168.100.20

;; Query time: 0 msec
;; SERVER: 192.168.100.11#53(192.168.100.11) (UDP)
;; WHEN: Tue Dec 02 19:52:39 UTC 2025
;; MSG SIZE rcvd: 88

```

Figure 26. from server 3 to server 2 for the server

```

ubuntu@ubuntu-cloud:~$ dig @192.168.100.10 example.lab SOA

; <<>> DiG 9.18.39-0ubuntu0.22.04.2-Ubuntu <<>> @192.168.100.10 example.lab SOA
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 39900
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 445e15b6160708a181000000692f43b26314182b6b19e22 (good)
;; QUESTION SECTION:
;example.lab.                    IN      SOA

;; ANSWER SECTION:
example.lab.                    86400   IN      SOA     ns1.example.lab. admin.example.lab. 2025120202 3600 0

;; Query time: 3 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Tue Dec 02 19:53:22 UTC 2025
;; MSG SIZE rcvd: 114

```

```

ubuntu@ubuntu-cloud:~$ dig @192.168.100.10 www.example.lab

; <<>> DiG 9.18.39-0ubuntu0.22.04.2-Ubuntu <<>> @192.168.100.10 www.example.lab
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 6291
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 005305b321958b1101000000692f44ea680c546a0d21ca7e (good)
;; QUESTION SECTION:
;www.example.lab.                IN      A

;; ANSWER SECTION:
www.example.lab.                86400   IN      A      192.168.100.20

;; Query time: 7 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Tue Dec 02 19:58:34 UTC 2025
;; MSG SIZE rcvd: 88

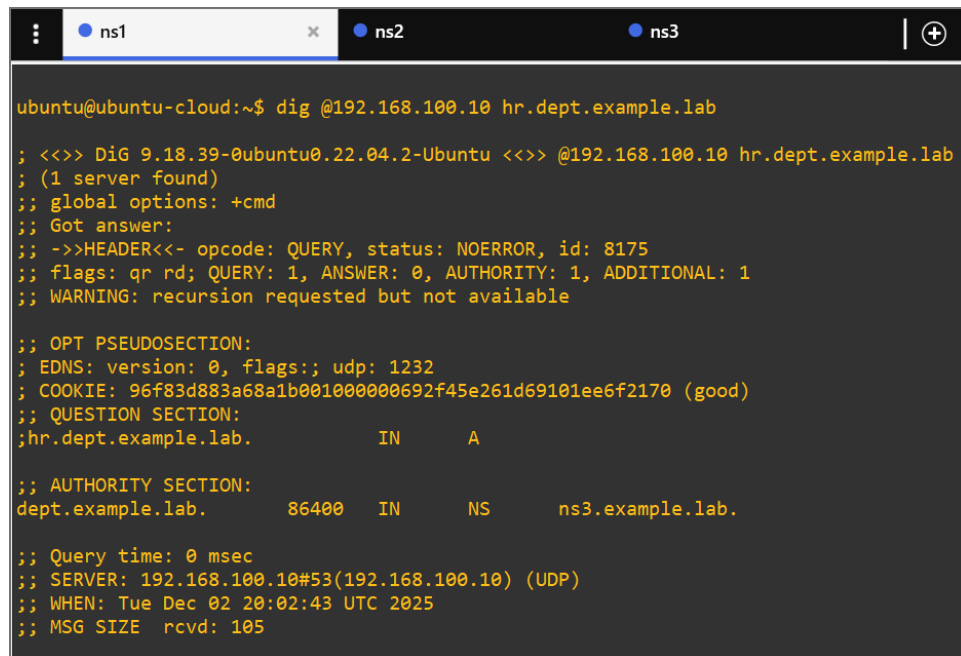
```

Figure 27. from server 2 to server 1 for the SOA record and the server

Task 5.2: Test Subdomain Delegation

I queried the delegated subdomain: those to the parent show referral to ns3, while queries to ns3 return authoritative answers.

Figures are bellow.



```
ubuntu@ubuntu-cloud:~$ dig @192.168.100.10 hr.dept.example.lab

; <<>> DiG 9.18.39-0ubuntu0.22.04.2-Ubuntu <<>> @192.168.100.10 hr.dept.example.lab
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 8175
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 1232
;; COOKIE: 96f83d883a68a1b001000000692f45e261d69101ee6f2170 (good)
;; QUESTION SECTION:
;hr.dept.example.lab.          IN      A

;; AUTHORITY SECTION:
dept.example.lab.      86400   IN      NS      ns3.example.lab.

;; Query time: 0 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Tue Dec 02 20:02:43 UTC 2025
;; MSG SIZE rcvd: 105
```

- I queried subdomain from parent server:

```
dig @192.168.100.10 hr.dept.example.lab
```

```
ubuntu@ubuntu-cloud:~$ dig @192.168.100.12 hr.dept.example.lab

; <<>> DiG 9.18.39-0ubuntu0.22.04.2-Ubuntu <<>> @192.168.100.12 hr.dept.example.lab
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 62523
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 45364d3f1419db6801000000692f473e32763c10bfe3c249 (good)
;; QUESTION SECTION:
;hr.dept.example.lab.          IN      A

;; ANSWER SECTION:
hr.dept.example.lab.      86400   IN      A      192.168.100.50

;; Query time: 4 msec
;; SERVER: 192.168.100.12#53(192.168.100.12) (UDP)
;; WHEN: Tue Dec 02 20:08:31 UTC 2025
;; MSG SIZE rcvd: 92
```

- Then I queried subdomain from delegated server

```
dig @192.168.100.12 hr.dept.example.lab
```

```
ubuntu@ubuntu-cloud:~$ dig @192.168.100.10 dept.example.lab NS

; <<>> DiG 9.18.39-0ubuntu0.22.04.2-Ubuntu <<>> @192.168.100.10 dept.example.lab NS
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 26536
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 68778b980c03451101000000692f475ddb95eaa56030702 (good)
;; QUESTION SECTION:
;dept.example.lab.          IN      NS

;; AUTHORITY SECTION:
dept.example.lab.      86400   IN      NS      ns3.example.lab.

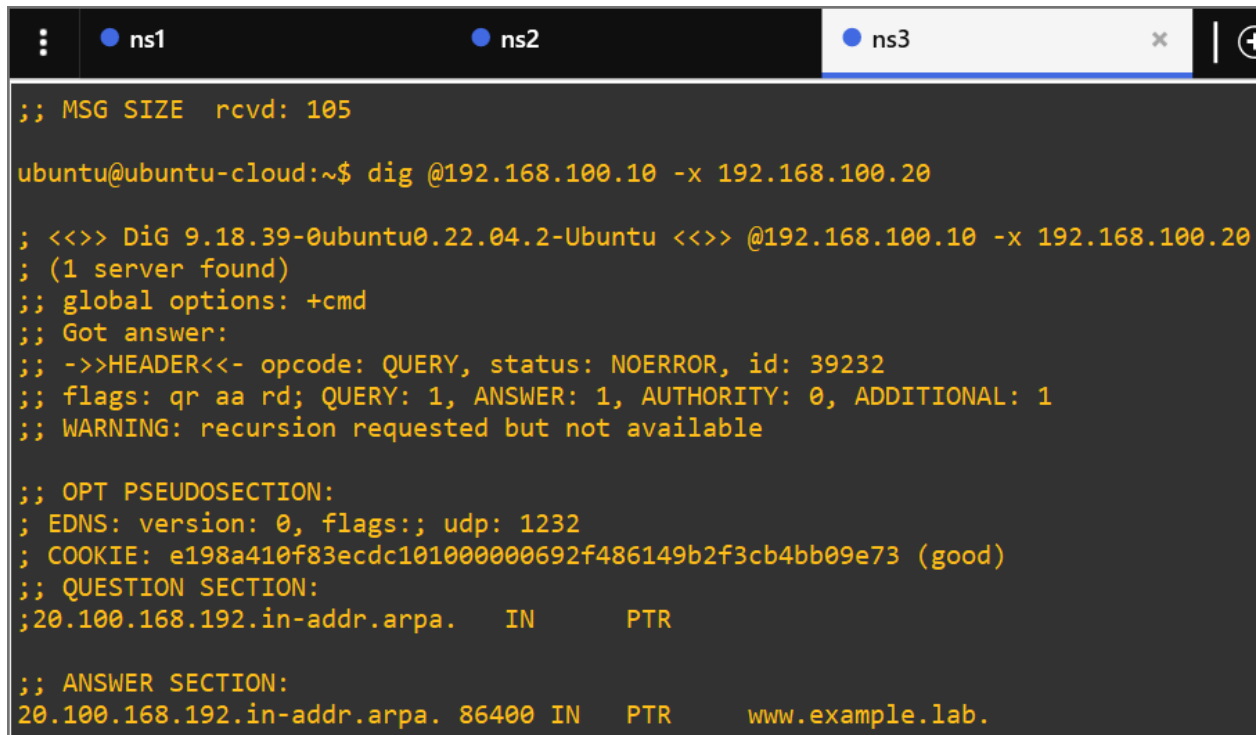
;; Query time: 4 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (UDP)
;; WHEN: Tue Dec 02 20:09:01 UTC 2025
;; MSG SIZE rcvd: 91
```

- At the end I checked NS records for subdomain

```
dig @192.168.100.10 dept.example.lab NS
```

Task 5.3: Test Reverse DNS

I did reverse lookup from server 3,2,1 and got example.lab - Success of the check, figure 28.



```
;; MSG SIZE rcvd: 105

ubuntu@ubuntu-cloud:~$ dig @192.168.100.10 -x 192.168.100.20

; <<>> DiG 9.18.39-0ubuntu0.22.04.2-Ubuntu <<>> @192.168.100.10 -x 192.168.100.20
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 39232
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

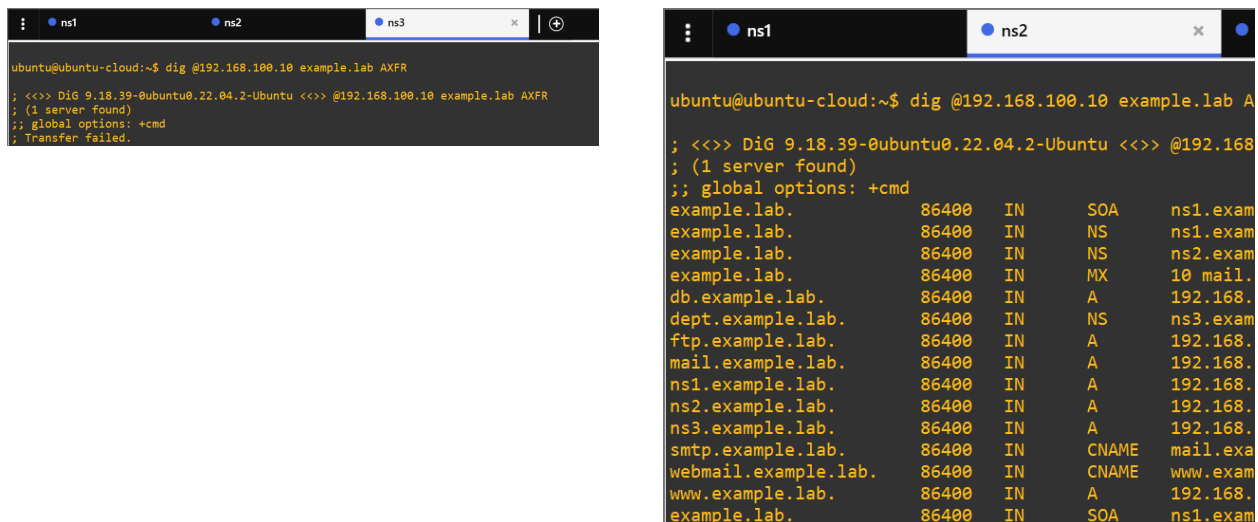
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: e198a410f83ecdc101000000692f486149b2f3cb4bb09e73 (good)
;; QUESTION SECTION:
;20.100.168.192.in-addr.arpa.    IN      PTR

;; ANSWER SECTION:
20.100.168.192.in-addr.arpa. 86400 IN      PTR      www.example.lab.
```

Figure 28. Reverse lookup from server 3.

Task 5.4: Test Zone Transfer Security

I attempted zone transfer from an unauthorized host (server 3). Transfer from unauthorized server 3 failed and from server 2 succeeded, figure 29.



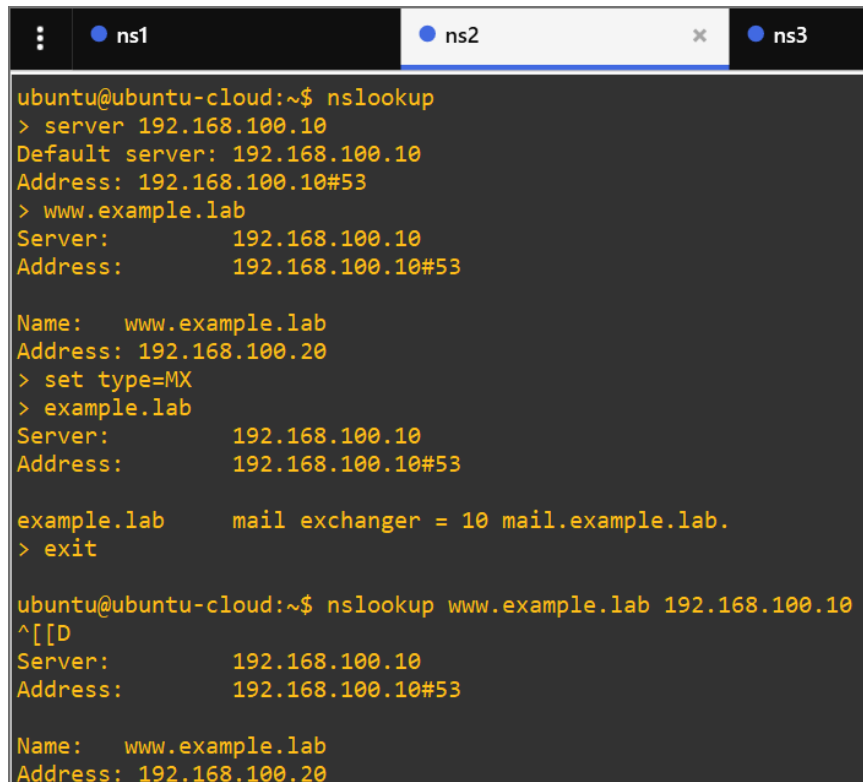
```
ubuntu@ubuntu-cloud:~$ dig @192.168.100.10 example.lab AXFR
; <<>> DiG 9.18.39-0ubuntu0.22.04.2-Ubuntu <<>> @192.168.100.10 example.lab AXFR
; (1 server found)
;; global options: +cmd
; Transfer failed.

ubuntu@ubuntu-cloud:~$ dig @192.168.100.10 example.lab A
; <<>> DiG 9.18.39-0ubuntu0.22.04.2-Ubuntu <<>> @192.168.100.10 example.lab A
; (1 server found)
;; global options: +cmd
example.lab.      86400    IN       SOA      ns1.exam
example.lab.      86400    IN       NS       ns1.exam
example.lab.      86400    IN       NS       ns2.exam
example.lab.      86400    IN       MX       10 mail.
db.example.lab.   86400    IN       A        192.168.
dept.example.lab. 86400    IN       NS       ns3.exam
ftp.example.lab.  86400    IN       A        192.168.
mail.example.lab. 86400    IN       A        192.168.
ns1.example.lab.  86400    IN       A        192.168.
ns2.example.lab.  86400    IN       A        192.168.
ns3.example.lab.  86400    IN       A        192.168.
smtp.example.lab. 86400    IN       CNAME    mail.exa
webmail.example.lab. 86400    IN       CNAME    www.exa
www.example.lab.  86400    IN       A        192.168.
example.lab.      86400    IN       SOA      ns1.exam
```

Figure 29. Log of attempts to dig from AXFR (from server 3,2)

Task 5.5: Using nslookup for Testing

I used `nslookup` for Testing in 2 modes and **all successfully show right results**, figure 30.



```
ubuntu@ubuntu-cloud:~$ nslookup
> server 192.168.100.10
Default server: 192.168.100.10
Address: 192.168.100.10#53
> www.example.lab
Server:          192.168.100.10
Address:         192.168.100.10#53

Name:   www.example.lab
Address: 192.168.100.20
> set type=MX
> example.lab
Server:          192.168.100.10
Address:         192.168.100.10#53

example.lab      mail exchanger = 10 mail.example.lab.
> exit

ubuntu@ubuntu-cloud:~$ nslookup www.example.lab 192.168.100.10
^[[D
Server:          192.168.100.10
Address:         192.168.100.10#53

Name:   www.example.lab
Address: 192.168.100.20
```

Figure 29. Log of attempts using `nslookup`

Task 5.6: DNS Trace Testing

For this task I traced the full resolution path:

```
dig @192.168.100.10 hr.dept.example.lab +trace
```

Figure 30 shows the complete delegation chain from root servers to your subdomain.

```

gns3@gns3vm:~$ dig @192.168.100.10 hr.dept.example.lab +trace
; <<> DiG 9.18.30-0ubuntu0.20.04.2-Ubuntu <<> @192.168.100.10 hr.dept.example.lab +trace
; (1 server found)
;; global options: +cmd
.      2657    IN      NS      d.root-servers.net.
.      2657    IN      NS      e.root-servers.net.
.      2657    IN      NS      f.root-servers.net.
.      2657    IN      NS      g.root-servers.net.
.      2657    IN      NS      h.root-servers.net.
.      2657    IN      NS      i.root-servers.net.
.      2657    IN      NS      j.root-servers.net.
.      2657    IN      NS      k.root-servers.net.
.      2657    IN      NS      l.root-servers.net.
.      2657    IN      NS      m.root-servers.net.
.      2657    IN      NS      a.root-servers.net.
.      2657    IN      NS      b.root-servers.net.
.      2657    IN      NS      c.root-servers.net.
.      2657    IN      RRSIG   NS 8 0 518400 20251215170000 20251202160000 61809 . h9n5tkY8Vo66HgBk3mQV1dLVZXR2
mHbarBRrH50vyNjEuwwx5sw19gEp mfPoE1jkY/hD1c9Xsky9dT5RrLi7c0Uuc/ev+9P7wbR0aefwGvEVS6 XY1WjYyMyok3g3XJNhcZ5Q5PbdbAnwC2HNmYkyYxxB
M71sremc+QUY11 Qe40I9PWWAwXKGe+wc4xCRZc2IKUDrJtfmsyVovxq+h8Fo/2QnU2m75A P4G+Uuk6bb0pkKYVV1bv6ognDeqxyP0JcD4xmPgtFLVEN1DWia2C1Vzq
sw1VhIdWpmmPm1aSi3V22gJBP5GGFzX//KffABnXBJUERTf8wgYm6MR1 K6ktbQ==
;; Received 525 bytes from 192.168.100.10#53(192.168.100.10) in 28 ms

```

Figure 30. Trace log

Task 5.7: Check DNS Server Logs

Monitoring of DNS activity is done and showed in figures 31,32.

```

ubuntu@ubuntu-cloud:~$ sudo tail -f /var/log/syslog | grep named
Dec  2 20:15:13 ubuntu-cloud named[818]: client @0x7fec81298978 192.168.100.12#35477 (example.lab): z
one transfer 'example.lab/AXFR/IN' denied
Dec  2 20:15:17 ubuntu-cloud named[818]: client @0x7fec81298978 192.168.100.11#33599 (example.lab): t
ransfer of 'example.lab/IN': AXFR started (serial 2025120202)
Dec  2 20:15:17 ubuntu-cloud named[818]: client @0x7fec81298978 192.168.100.11#33599 (example.lab): t
ransfer of 'example.lab/IN': AXFR ended: 1 messages, 15 records, 390 bytes, 0.001 secs (390000 bytes/
sec) (serial 2025120202)
Dec  2 20:15:21 ubuntu-cloud named[818]: client @0x7fec81298978 192.168.100.10#33021 (example.lab): z
one transfer 'example.lab/AXFR/IN' denied

```

Figure 31. Real-time log monitoring

```
sudo tail -f /var/log/syslog | grep named
```



The image shows a terminal window with three tabs labeled ns1, ns2, and ns3. The ns1 tab is active. The terminal displays the command `sudo grep "query" /var/log/syslog | tail -20` and its output, which consists of ten log entries. Each entry shows a date and time (Dec 2), a timestamp, the hostname (ubuntu-cloud), the process name (named), and the message (resolver priming query complete: failure).

```
^C
ubuntu@ubuntu-cloud:~$ sudo grep "query" /var/log/syslog | tail -20
Dec  2 16:20:18 ubuntu-cloud named[18216]: resolver priming query complete: failure
Dec  2 16:29:26 ubuntu-cloud named[579]: resolver priming query complete: failure
Dec  2 17:20:18 ubuntu-cloud named[579]: resolver priming query complete: failure
Dec  2 17:29:26 ubuntu-cloud named[579]: resolver priming query complete: failure
Dec  2 17:50:29 ubuntu-cloud named[1108]: resolver priming query complete: failure
Dec  2 18:29:26 ubuntu-cloud named[1108]: resolver priming query complete: failure
Dec  2 18:56:08 ubuntu-cloud named[1143]: resolver priming query complete: failure
Dec  2 19:35:55 ubuntu-cloud named[579]: resolver priming query complete: failure
Dec  2 19:50:38 ubuntu-cloud named[818]: resolver priming query complete: failure
```

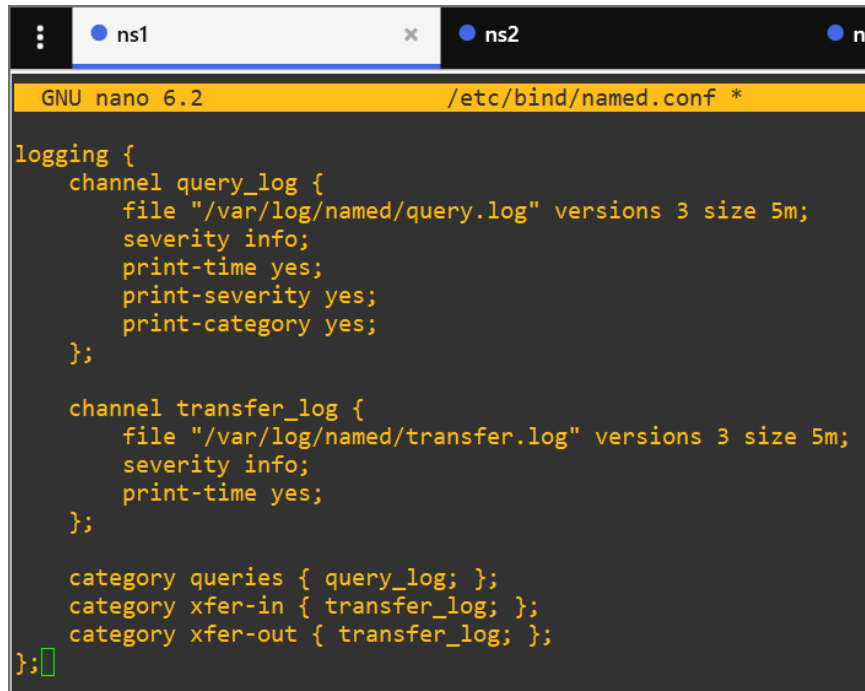
Figure 32. Search result for specific queries

```
sudo grep "query" /var/log/syslog | tail -20
```

Part 6: Advanced Configuration Tasks

Task 6.1: Configure DNS Logging

On **Server 1**, I enhanced logging capabilities, editing the main configuration, shown in the figure 33.



```
GNU nano 6.2 /etc/bind/named.conf *
logging {
    channel query_log {
        file "/var/log/named/query.log" versions 3 size 5m;
        severity info;
        print-time yes;
        print-severity yes;
        print-category yes;
    };

    channel transfer_log {
        file "/var/log/named/transfer.log" versions 3 size 5m;
        severity info;
        print-time yes;
    };

    category queries { query_log; };
    category xfer-in { transfer_log; };
    category xfer-out { transfer_log; };
};
```

Figure 33. Main configuration for logging.

Next step was to create log directory, setup access policy and restart the service:

```
sudo mkdir -p /var/log/named
```

```
sudo chown bind:bind /var/log/named
```

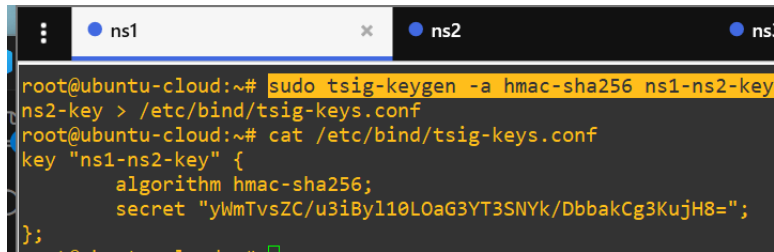
```
sudo systemctl restart bind9
```

Task 6.2: Implement TSIG for Secure Zone Transfers

Firstly I generated the key, figure 34.

```
sudo tsig-keygen -a hmac-sha256 ns1-ns2-key > /etc/bind/tsig-keys.conf
```

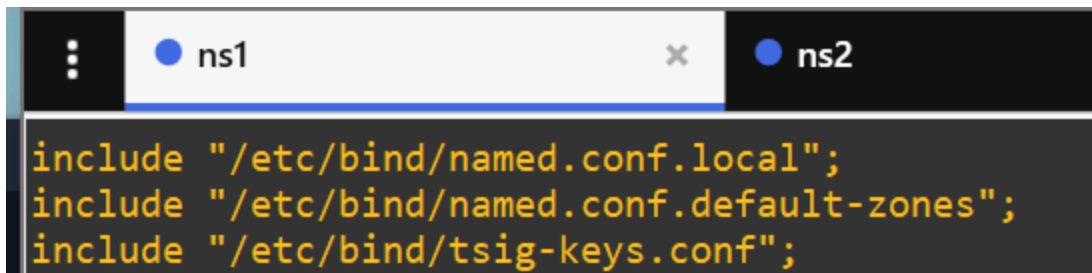
Output example

A terminal window with tabs for ns1 and ns2. The command 'sudo tsig-keygen -a hmac-sha256 ns1-ns2-key' is executed, creating a file at /etc/bind/tsig-keys.conf. The 'cat' command is used to display the contents of this file, which shows a key definition for 'ns1-ns2-key' using the 'hmac-sha256' algorithm and a specific secret string.

```
root@ubuntu-cloud:~# sudo tsig-keygen -a hmac-sha256 ns1-ns2-key
ns2-key > /etc/bind/tsig-keys.conf
root@ubuntu-cloud:~# cat /etc/bind/tsig-keys.conf
key "ns1-ns2-key" {
    algorithm hmac-sha256;
    secret "yWmTvsZC/u3iByl10LOaG3YT3SNYk/DbbakCg3KujH8=";
};
```

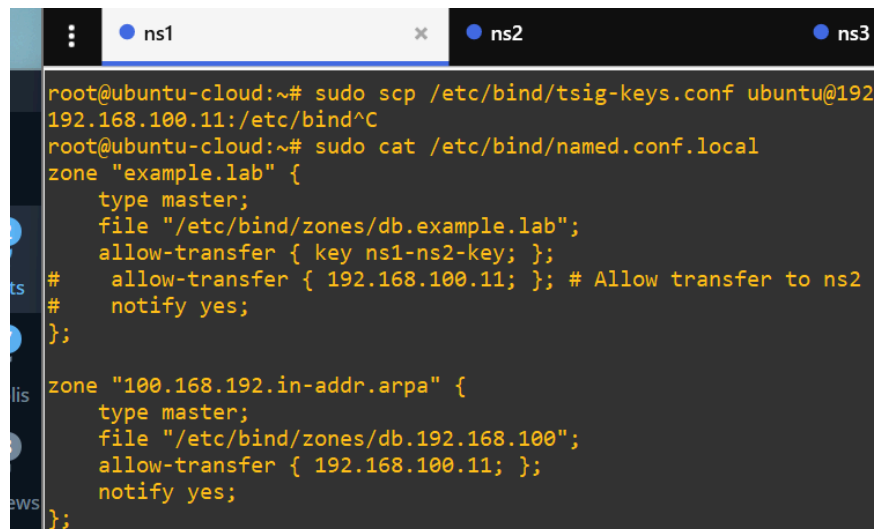
Figure 34. Key is generated and shown, using `cat` command

Then I included the key in named.conf and in zone definition files, figure 35,36.

A terminal window with tabs for ns1 and ns2. The 'cat' command is used to show the contents of /etc/bind/named.conf.local, which includes the file /etc/bind/tsig-keys.conf.

```
include "/etc/bind/named.conf.local";
include "/etc/bind/named.conf.default-zones";
include "/etc/bind/tsig-keys.conf";
```

Figure 35. named.conf files with includes

A terminal window with tabs for ns1, ns2, and ns3. The 'scp' command is used to copy the tsig-keys.conf file to server 2 (192.168.100.11). The 'cat' command is used to show the contents of /etc/bind/named.conf.local, which defines two zones: 'example.lab' and '100.168.192.in-addr.arpa'. Both zones are configured as master zones and include the key 'ns1-ns2-key' for allow-transfer.

```
root@ubuntu-cloud:~# sudo scp /etc/bind/tsig-keys.conf ubuntu@192.168.100.11:/etc/bind^C
root@ubuntu-cloud:~# sudo cat /etc/bind/named.conf.local
zone "example.lab" {
    type master;
    file "/etc/bind/zones/db.example.lab";
    allow-transfer { key ns1-ns2-key; };
    # allow-transfer { 192.168.100.11; }; # Allow transfer to ns2
    # notify yes;
};

zone "100.168.192.in-addr.arpa" {
    type master;
    file "/etc/bind/zones/db.192.168.100";
    allow-transfer { 192.168.100.11; };
    notify yes;
};
```

Figure 36. zone definition file updated and `scp` action to server 2

Then on server 2, I copied and moved the same TSIG key file with `scp` and updated configuration, figure 37.

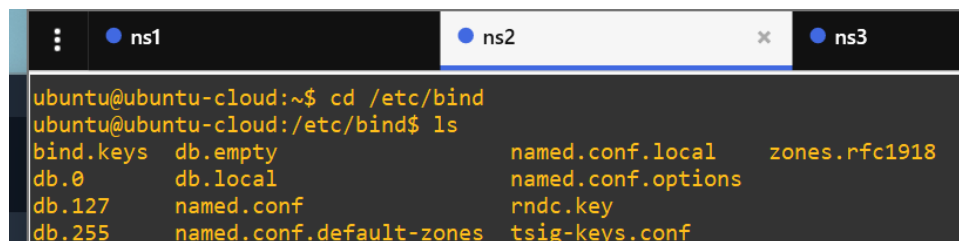
I used `sudo nano /etc/bind/named.conf` and added:

```
include "/etc/bind/tsig-keys.conf";

server 192.168.100.10 {

    keys { ns1-ns2-key; };

};
```

A terminal window with three tabs labeled 'ns1', 'ns2', and 'ns3'. The 'ns2' tab is active. The terminal shows the following commands and output:

```
ubuntu@ubuntu-cloud:~$ cd /etc/bind
ubuntu@ubuntu-cloud:/etc/bind$ ls
bind.keys  db.empty      named.conf.local  zones.rfc1918
db.0       db.local      named.conf.options
db.127     named.conf    rndc.key
db.255     named.conf.default-zones  tsig-keys.conf
```

Figure 37. File is moved

To refresh changes I restarted both servers and test zone transfer with TSIG, figure 38.

```
ns1 ns2 ns3
root@ubuntu-cloud:~# dig @192.168.100.10 example.lab AXFR

; <<>> DiG 9.18.39-0ubuntu0.22.04.2-Ubuntu <<>> @192.168.100.10 example.lab AXFR
; (1 server found)
;; global options: +cmd
example.lab.      86400 IN      SOA      ns1.example.lab. admin.example.lab. 2025120201 3600 1800 604800 86400
example.lab.      86400 IN      NS       ns1.example.lab.
example.lab.      86400 IN      NS       ns2.example.lab.
example.lab.      86400 IN      MX       10 mail.example.lab.
db.example.lab.   86400 IN      A        192.168.100.35
dept.example.lab. 86400 IN      NS       ns3.example.lab.
ftp.example.lab.  86400 IN      A        192.168.100.30
mail.example.lab. 86400 IN      A        192.168.100.25
ns1.example.lab.  86400 IN      A        192.168.100.10
ns2.example.lab.  86400 IN      A        192.168.100.11
ns3.example.lab.  86400 IN      A        192.168.100.12
smtp.example.lab. 86400 IN      CNAME    mail.example.lab.
webmail.example.lab. 86400 IN      CNAME    www.example.lab.
www.example.lab.  86400 IN      A        192.168.100.20
example.lab.      86400 IN      SOA      ns1.example.lab. admin.example.lab. 2025120201 3600 1800 604800 86400
;; Query time: 4 msec
;; SERVER: 192.168.100.10#53(192.168.100.10) (TCP)
;; WHEN: Tue Dec 02 19:06:31 UTC 2025
;; XFR size: 15 records (messages 1, bytes 390)
```

Figure 37. Test between server 2 and server 1

6. Questions and Analysis

Question 1: Explain the difference between authoritative and recursive DNS servers. Which type did you configure in this lab?

Authoritative server → serves only the data it owns. No recursion. Responds with final answers from its zone files.

Recursive server → performs full resolution on behalf of clients by querying other DNS servers.

In this lab, the configuration explicitly set `recursion no`, used local zone files, and served authoritative data only. So the servers (ns1, ns2, ns3) were **authoritative DNS servers**, not recursive ones.

Question 2: What is the purpose of the Serial number in the SOA record? What happens if you forget to increment it when making zone changes?

Serial number tracks the version of a zone. And secondary servers use it to determine whether they must fetch a new copy.

If one forgets to increment it, the master has newer data, but secondaries believe nothing changed.

Result: **clients continue receiving stale data** from secondaries.

Takeaway: updates exist, but no propagation happens.

Question 3: Describe the zone transfer process. What are the differences between AXFR and IXFR?

1. **Zone transfer** is a mechanism for synchronizing zone data between master and secondary.
2. **AXFR** → full zone transfer (complete copy each time).
3. **IXFR** → incremental transfer (only differences based on serial deltas).

AXFR is simple but wastes bandwidth; IXFR is efficient but requires journaling and cooperation of both servers.

Question 4: Why are glue records necessary in DNS delegation? What problems occur without them?

- **DNS delegation** is a process of moving authority from parent zone to downwards.
- **Glue record** is a record for a nameserver that lives inside the delegated zone.
- Required because the parent zone must give the IP of the delegated NS to avoid circular lookups.

Without glue, the resolver asks the parent for the NS, then must ask those NS servers for their own IPs—which is impossible because it cannot reach them

without already knowing their IPs.

Problems: **resolution loop or complete delegation failure.**

My glue was the `ns3 IN A 192.168.100.12` entry in `example.lab`.

Question 5: What security risks exist with improperly configured zone transfers? How did you mitigate these risks in your configuration?

- Unrestricted zone transfers expose the full zone contents: hostnames, internal IPs, topology.
- Attackers can exploit this vulnerability for network mapping, phishing and etc.

I mitigated the risk using:

- `allow-transfer { 192.168.100.11; };` restricting AXFR to ns2;
- TSIG keys (`ns1-ns2-key`) for authenticated transfers.

And I got combined effect: **only ns2 can transfer** and only with the correct shared secret.

Question 6: If the primary DNS server fails, can clients still resolve names in the zone? Explain why or why not.

Yes, client can resolve, because secondary (slave) servers host a full replicated copy of the zone.

Any NS listed in the parent zone file is authoritative and valid to use. Clients query ns2 directly once ns1 becomes unreachable.

But, if secondary never successfully transferred the data, it eventually expires after the SOA "Expire" timer and stops answering.

Question 7: Describe three methods you used to troubleshoot DNS issues during this lab.

- **dig queries** (A, MX..., +trace, AXFR) to verify records, delegation, and zone consistency.

```
dig @192.168.100.10 hr.dept.example.lab +trace
```

- **Log inspection** via `/var/log/syslog` + `| grep named` to observe chosen queries.

```
sudo tail -f /var/log/syslog | grep named
```

- **Configuration validation** using `named-checkconf` and `named-checkzone` to catch syntax errors before restart.

```
sudo named-checkzone example.lab /etc/bind/zones/db.example.lab
```

Question 8: What is the purpose of TSIG? In what scenarios would you implement it in a production environment?

TSIG provides cryptographic authentication for DNS operations (typically zone transfers and dynamic updates). It prevents unauthorized hosts from initiating AXFR/IXFR or poisoning dynamic update zones.

Production scenarios where TSIG is mandatory:

- Master–secondary synchronization across untrusted networks
- Environments requiring signed dynamic updates (DHCP + DNS integration)

- Any setup where zone-transfer leakage is a security risk

Takeaway: TSIG ensures **integrity + authenticity** for DNS communications.

Takeaways from the work done:



- Done: authoritative-only DNS deployed, with recursion disabled and direct control over zone data.
- New: Serial numbers controlled propagation, and forgetting them would silently break synchronization.
- Done: zone transfers were secured by strict ACLs and TSIG to prevent data leakage.
- New: troubleshooting depended on `dig`, logs, and strict configuration validation.
- Never change the name of projects outside GNS interface, otherwise configuration disappears 🦴

8. References

1. Ubuntu official site, download pages. Link <https://ubuntu.com/download/server/thank-you?version=24.04.3&architecture=amd64&its=true>
2. RFC 1035 – Domain Names: Implementation and Specification. Link: <https://datatracker.ietf.org/doc/html/rfc1035>
3. RFC 1034 – Domain Names: Concepts and Facilities. Link: <https://datatracker.ietf.org/doc/html/rfc1034>

4. TSIG: Secret Key Transaction Authentication for DNS. Link:
<https://datatracker.ietf.org/doc/html/rfc2845>
5. ISC BIND 9 Knowledge Base (DNS Operations & Tutorials). Link:
<https://kb.isc.org/>
6. RFC 5936 – DNS Zone Transfer Protocol (AXFR/IXFR). Link:
<https://datatracker.ietf.org/doc/html/rfc5936>